

Solution 1: Multi-Channel Notification System

Pattern used: Factory Pattern + Singleton

```
interface Notification {
    void send();
}

class EmailService implements Notification {
    private static EmailService instance;
    private EmailService() {}
    public static EmailService getInstance() {
        if (instance == null)
            instance = new EmailService();
        return instance;
    }
    public void send() {
        System.out.println("Email Notification Sent");
    }
}

class SMSService implements Notification {
    private static SMSService instance;
    private SMSService() {}
    public static SMSService getInstance() {
        if (instance == null)
            instance = new SMSService();
        return instance;
    }
    public void send() {
        System.out.println("SMS Notification Sent");
    }
}

class NotificationFactory {
    public static Notification getNotification(String type) {
        if (type.equalsIgnoreCase("email"))
            return EmailService.getInstance();
        else
            return SMSService.getInstance();
    }
}

class Day1Factory {
    public static void main(String[] args) {
        Notification n1 = NotificationFactory.getNotification("email");
        Notification n2 = NotificationFactory.getNotification("sms");
        n1.send();
        n2.send();
    }
}
```

Solution 2: Travel Itinerary Generation System

Pattern used: Builder Pattern + Singleton Configuration Manager

```
class ConfigManager {
    private static ConfigManager instance;
    private ConfigManager() {}
    public static ConfigManager getInstance() {
        if (instance == null)
            instance = new ConfigManager();
        return instance;
    }
}
```

```

        public int getBudget() { return 5000; }
    }

    class Itinerary {
        String hotel;
        String transport;
        void show() {
            System.out.println("Hotel: " + hotel);
            System.out.println("Transport: " + transport);
        }
    }

    abstract class ItineraryBuilder {
        Itinerary i = new Itinerary();
        abstract void buildHotel();
        abstract void buildTransport();
        Itinerary getItinerary() { return i; }
    }

    class LuxuryItineraryBuilder extends ItineraryBuilder {
        void buildHotel() { i.hotel = "5 Star Hotel"; }
        void buildTransport() { i.transport = "Flight"; }
    }

    class BudgetItineraryBuilder extends ItineraryBuilder {
        void buildHotel() { i.hotel = "3 Star Hotel"; }
        void buildTransport() { i.transport = "Bus"; }
    }

    class TravelDirector {
        ItineraryBuilder builder;
        TravelDirector(ItineraryBuilder builder) { this.builder = builder; }
        Itinerary construct() {
            builder.buildHotel();
            builder.buildTransport();
            return builder.getItinerary();
        }
    }

    class Demo {
        public static void main(String[] args) {
            ConfigManager config = ConfigManager.getInstance();
            System.out.println("Budget: " + config.getBudget());

            TravelDirector d1 = new TravelDirector(new LuxuryItineraryBuilder());
            d1.construct().show();

            System.out.println();

            TravelDirector d2 = new TravelDirector(new BudgetItineraryBuilder());
            d2.construct().show();
        }
    }

```